POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
College of Computer and Information Sciences
Sta. Mesa, Manila

GeneSync: Dynamic Programming-Based Optimization of Global Pairwise Sequence Alignment Using the Needleman–Wunsch Algorithm

Prepared by

Alpon, Rheinz Owen D.
Lozano, Mac Edison S.
Magalona, Adriel M.
Puti, Jude Vincent F.
Solomon, Nazareth Z.

Instructor: Montaigne G. Molejon

Date: June 19, 2026

I. Introduction and its Background

The rapid advancement of genomics, molecular biology, and bioinformatics has produced large amounts of biological sequence data, such as DNA, RNA, and protein sequences. These sequences contain important biological information that can be used to study genetic similarities, mutations, evolutionary relationships, and functional connections between organisms. Because of this, sequence comparison has become an important task in bioinformatics.

One fundamental method used in comparing biological sequences is pairwise sequence alignment. This process arranges two sequences in a way that shows their similarities and differences. It is commonly used for identifying conserved regions, detecting mutations, comparing genes, and studying evolutionary relationships. However, manually aligning two sequences can be difficult and inefficient. As the length of the sequences increases, the number of possible alignments also increases. This makes it impractical to check every possible alignment manually, especially when the sequences contain matches, mismatches, and gaps.

To address this problem, this study focuses on the Needleman–Wunsch algorithm, a dynamic programming algorithm used for global pairwise sequence alignment. The algorithm compares two sequences from beginning to end and determines the best possible alignment using a scoring system for matches, mismatches, and gaps. Through dynamic programming, the Needleman–Wunsch algorithm stores computed values in a matrix and reuses them to find the optimal alignment efficiently. This project aims to demonstrate how the algorithm initializes the scoring matrix, computes alignment scores, and performs traceback to reconstruct the final optimal global alignment.

Beyond the computational challenge of alignment, the true utility of the Needleman–Wunsch algorithm lies in the quantitative interpretation of its results. Simply generating an alignment is insufficient without metrics to determine if the similarities found are biologically meaningful or merely the result of random chance (Pearson, 2013). Consequently, this study implements automated statistical analysis to calculate the percentage of matching characters and evaluate if sequences align at a significant level.

Furthermore, sequence alignment serves as the critical first step in predicting the functional and phenotypic impact of genetic variations (Fernald et al., 2011). Because laboratory validation of genetic mutations is highly resource-intensive, computational frameworks increasingly rely on sequence alignment to assess evolutionary conservation; amino acids that remain unchanged across species are highly likely to be structurally vital, meaning mutations at these positions are frequently deleterious. By identifying the specific portions of the sequence containing the most gaps or mismatches, this implementation can pinpoint "mutational hotspots"—regions where structural changes such as insertions, deletions, or single-nucleotide variants are most prevalent (Choudhuri, 2014). This data-driven approach transforms raw sequence strings into a diagnostic report, establishing a foundation for downstream variant effect prediction and fulfilling the professional requirement for precision in genomic comparison.

II. Related Works

The study of Needleman, S. B. & Wunsch, C. D. (1970) was used as a guide in this study. Needleman and Wunsch introduced the first dynamic programming algorithm for global pairwise

sequence alignment of two biological sequences (protein or DNA). The original paper demonstrates how to construct a scoring matrix using a defined substitution score and a gap penalty, fill the matrix recursively by considering diagonal (match/mismatch), vertical (gap in sequence 1), and horizontal (gap in sequence 2) moves, and finally perform a traceback from the bottom-right cell to the top-left cell to retrieve the optimal alignment. A generalized flowchart of the Needleman–Wunsch algorithm is provided in their work.

Through this study, the researcher will apply dynamic programming concepts learned in the subject to properly initialize the matrix with progressive gap penalties and implement the traceback procedure in Python. The figure below is the Needleman–Wunsch global alignment algorithm based on their paper, which will be used for this study to find the optimal alignment between two DNA sequences.

The study of Mount, D. W. (2001), particularly the chapter on pairwise sequence alignment in *Bioinformatics: Sequence and Genome Analysis*, was also used as a guide. Mount provides a detailed step-by-step tutorial of the Needleman–Wunsch algorithm with a worked example on short nucleotide sequences (e.g., aligning GATTACA and GTCGACG). The chapter includes a clear diagram that illustrates the three-part process: (1) initialization of the matrix with gap penalties increasing linearly from the origin, (2) filling the matrix cell by cell using the recurrence relation, and (3) the traceback path marked with arrows showing the sequence of optimal moves. Through this study, the researcher will use this diagram as the structural blueprint for designing the program's output, ensuring that the scoring matrix and traceback can be visualized exactly as shown. This directly applies the subject's lessons on recurrence relations and optimal substructure, allowing the researcher to verify the correctness of the Python implementation at each step.

However, while these foundational texts provide the mechanical framework for alignment, modern research highlights the need for a more rigorous quantitative analysis of the results. According to Pearson (2013), the primary challenge in sequence alignment is distinguishing true biological homology from "random" similarity that occurs by chance. Furthermore, Choudhuri (2014) emphasizes that global alignment must be paired with an analysis of gap distribution to accurately pinpoint "mutational hotspots." These hotspots represent regions where significant structural shifts, such as large-scale insertions or deletions, are concentrated, providing deeper insight into evolutionary stability than a raw alignment score alone. By incorporating these analytical layers, this project moves beyond a standard implementation to address the modern professional requirement for precise genomic data interpretation.

III. Objectives of the Project

A. General Objective

The main objective of the project is to implement and demonstrate the Needleman–Wunsch algorithm for global pairwise sequence alignment using dynamic programming.

B. Specific Objectives

- To explain the purpose of global pairwise sequence alignment in comparing biological sequences.
- To apply the Needleman–Wunsch algorithm in aligning two sequences from beginning to end.

- To construct a dynamic programming matrix using a scoring system for matches, mismatches, and gaps.
- To compute the optimal alignment score by filling the matrix based on the Needleman–Wunsch recurrence relation.
- To perform traceback from the completed matrix in order to reconstruct the final optimal alignment.
- To calculate the percentage identity and evaluate the significance level of the alignment to provide a quantitative measure of genetic similarity.
- To identify segments with high gap density, allowing for the detection of specific mutational hotspots.
- To visualize the step-by-step process of the algorithm, including matrix initialization, matrix filling, traceback, and final alignment result.
- To show why Needleman–Wunsch is considered a dynamic programming algorithm.

IV. Scope and Limitations

This study focuses on the implementation and demonstration of the Needleman–Wunsch algorithm for global pairwise sequence alignment. The project will align two biological sequences from beginning to end using dynamic programming. It will show how the scoring matrix is created, how each cell is computed based on matches, mismatches, and gaps, and how traceback is performed to obtain the final optimal alignment.

The scope of this project includes:

1. Implementation of the Needleman–Wunsch algorithm only.
2. Alignment of two biological sequences globally by comparing them from start to end and inserting gaps when needed to make the sequences the same length.
3. Application of a scoring system for matches, mismatches, and gaps.
4. Generation of a statistical analysis report, which includes:
    - Percentage Identity: Determining the exact ratio of matching base pairs to the alignment length.
    - Significance Assessment: Checking if the alignment score exceeds thresholds that indicate biological relevance.
    - Gap Portion Mapping: Pinpointing the indices where gaps are most concentrated to identify structural mutations.
5. Visualization of the dynamic programming matrix and traceback process.
6. Generation of the final optimal alignment and alignment score.

This study is limited to global pairwise alignment and does not include local sequence alignment, multiple sequence alignment, or heuristic-based tools such as BLAST. The project will also not focus on large-scale genome analysis, biological interpretation of real medical data, or advanced scoring models. Its main purpose is to demonstrate how dynamic programming is applied in finding the optimal global alignment between two sequences using the Needleman–Wunsch algorithm.

The system also observes the following operational limitations:

1. **Input Pre-formatting:** Files must strictly conform to standard `.txt` or `.fasta` schemas. Any corrupted file headers, such as a missing > symbol in FASTA files, will halt execution.
2. **String Length Caps (Memory Guardrail):** Because the Needleman–Wunsch algorithm has a space and time complexity of $O(N \times M)$, the system must limit input sizes, such as allowing a maximum of 5,000 characters per sequence. Attempting to upload an unsegmented, full-length human chromosome file will cause an out-of-memory crash.
3. **Alphabet Restrictions:** The system only accepts valid biological characters:
   - **DNA:** A, T, C, G, N, where N represents an unknown or blurry base.
   - **RNA:** A, U, C, G.
   - **Proteins:** The 20 standard single-letter amino acid codes.
4. **No Multiple Sequence Alignment (MSA):** The system is strictly a pairwise alignment tool; it cannot accept three or more files simultaneously.

V. System Design and Methodology

GeneSync is designed as a sequential processing system that accepts two biological sequences and produces an optimal global alignment together with supporting statistical and visual results. Its architecture separates input preparation, dynamic programming, result analysis, and output presentation so that each stage of the alignment process can be observed and verified.

A. System Architecture

The system architecture is organized into four phases that follow the complete movement of data from file ingestion to final report export. The diagram below shows the major processing layers, their internal algorithms, and the data handoffs between them.

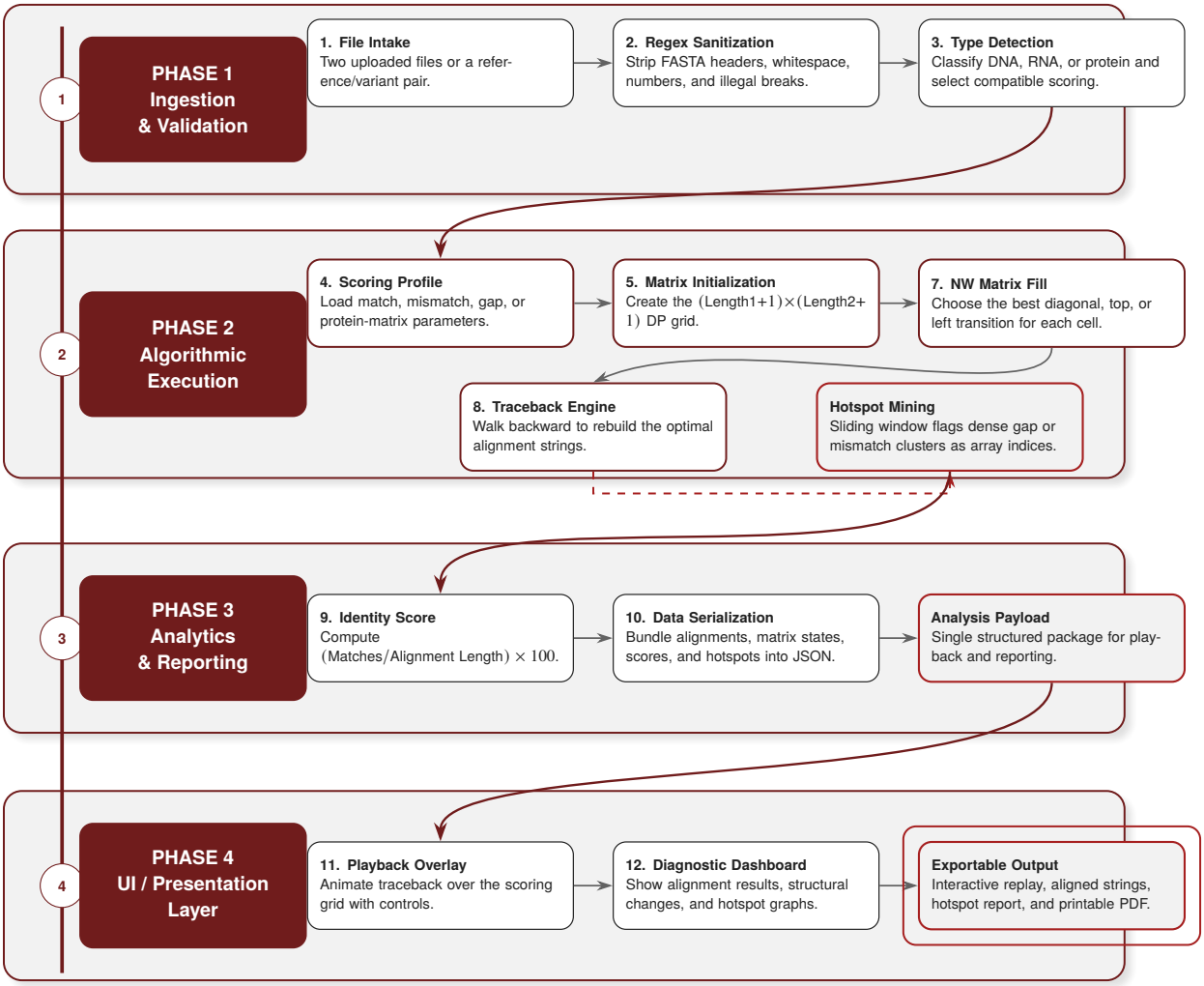


Figure 1: Four-phase GeneSync system architecture and algorithmic data-flow model

The system follows the step-by-step pipeline below:

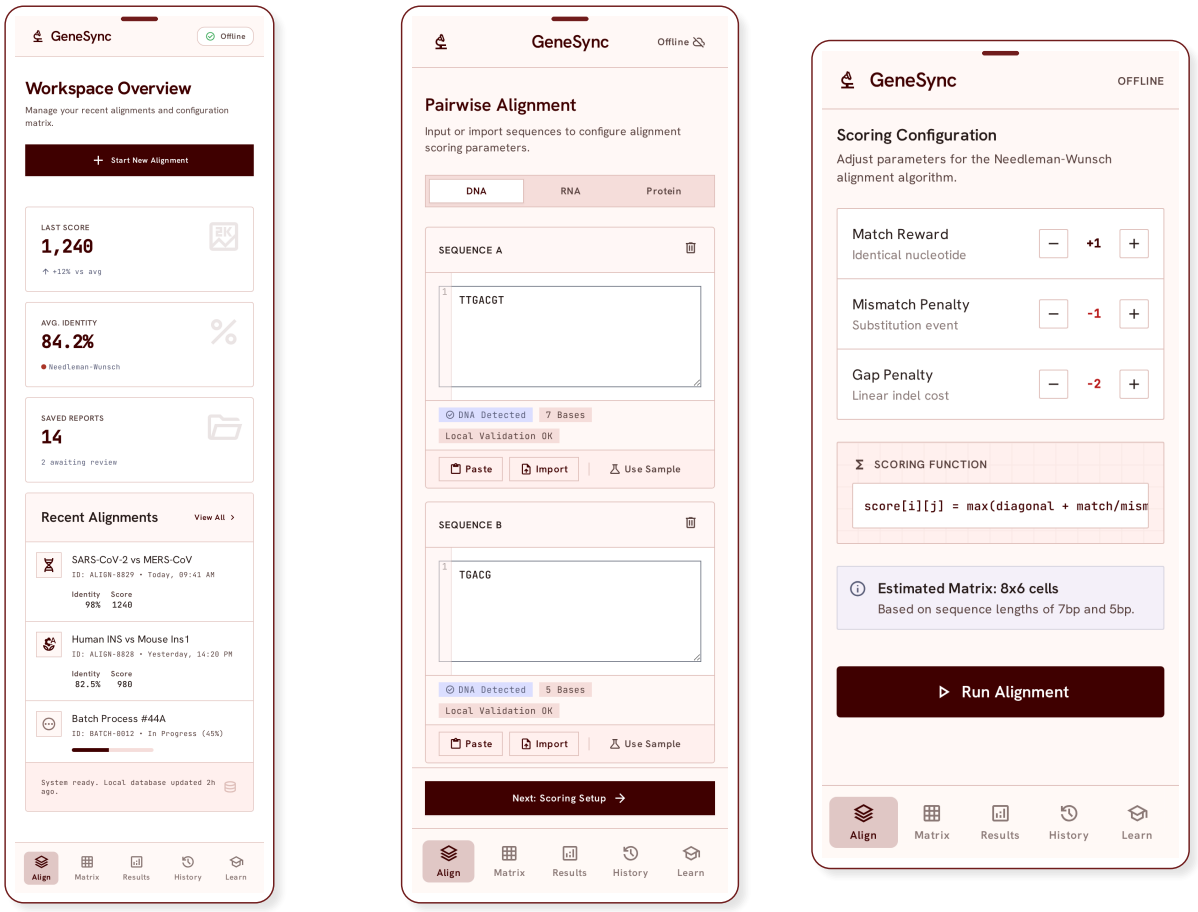
1. **Sequence Input and Validation (Acquisition Layer):** The user provides two DNA, RNA, or protein sequences and the corresponding scoring profile. The **Validation Gateway** strips FASTA structural headers via a **regular expression (regex) parser** and normalizes the alphabet strings. A sequence type scanner maps the input to verify valid symbols before execution begins.
2. **Scoring Matrix Initialization (Alignment Processing Core):** A two-dimensional memory array with $(m + 1) \times (n + 1)$ cells is initialized, where m and n denote sequence lengths. The **Matrix Initialization** module populates the boundary conditions, which are the first row and column, using cumulative linear gap penalties.
3. **Dynamic Programming Matrix Filling:** The **DP Scoring Kernel** computes the optimal score for every remaining cell. It evaluates the maximum value across a three-way recurrence relation: a diagonal step for match or mismatch, an upward step for a gap in sequence 1, and a leftward step for a gap in sequence 2, storing the result in the **State Matrix**.
4. **Traceback and Alignment Reconstruction:** Starting from the terminal bottom-right cell

$F(m, n)$, the **Traceback Engine** performs pathfinding backward to the origin $F(0, 0)$. It tracks the predecessor moves to reconstruct the optimal global alignment, translating diagonal steps into symbol alignments and axial steps into gap insertions.

5. **Statistical and Gap Analysis (Analytics & Interpretation):** The reconstructed alignment is parsed through a parallel analytics suite:
   - The **Identity Analyzer** applies a character summation algorithm to determine the exact percentage identity.
   - The **Significance Evaluator** cross-references the raw alignment score against threshold parameters via an empirical **Sequence Shuffling Algorithm** to rule out random chance similarity.
   - The **Hotspot Mapper** implements a **Sliding Window Algorithm** to monitor localized mismatch and gap frequencies, mapping out coordinates of structural instability, or "mutational hotspots."
6. **Visualization and Report Generation (Delivery Layer):** The **Report Assembly Service** packages the alignment sequences, dynamic programming cell grids, identified hotspots, and statistical parameters. The **Presentation Layer** uses a rendering engine to provide an animated playback overlay of the traceback matrix alongside the console diagnostics.

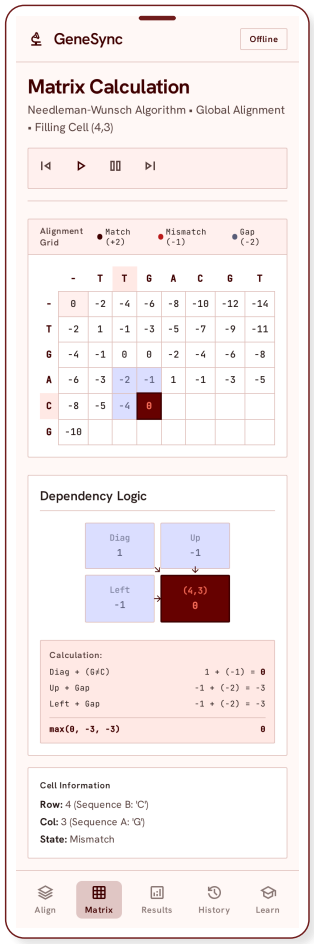
B. User Interface Mockups

The user interface mockups were prepared in Google Stitch to show how the system architecture is translated into a mobile workflow. The screens follow the same order as the processing pipeline: users start from a dashboard, provide and validate sequence data, configure the scoring values, run the Needleman–Wunsch matrix process, inspect the traceback path, and review the computed alignment and hotspot findings.

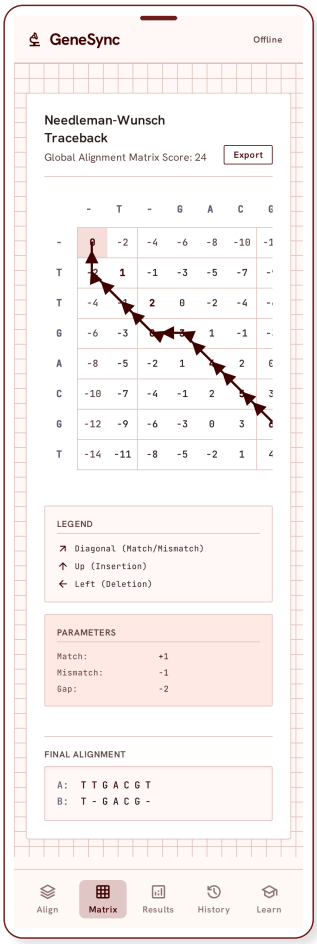


A. Dashboard overview **B. Pairwise sequence input** **C. Scoring configuration**

Figure 2: Google Stitch mockups for GeneSync input preparation and scoring setup

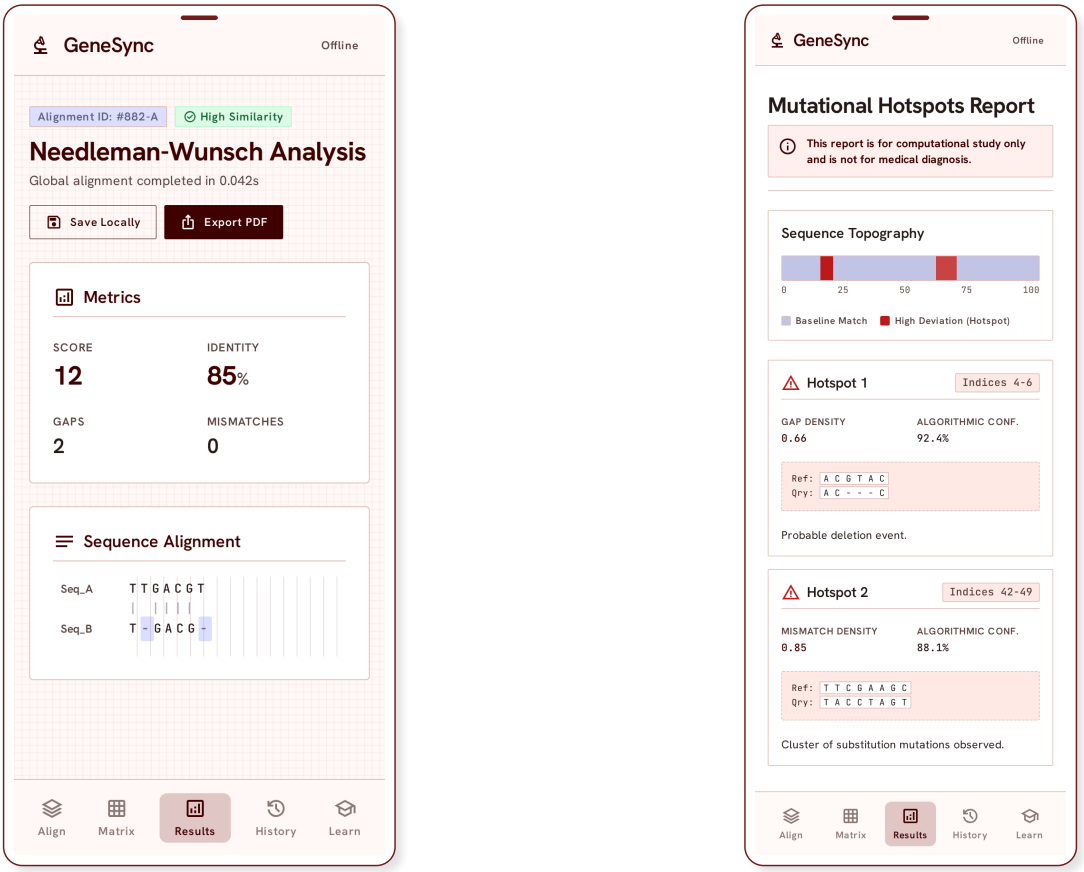


A. Matrix filling



B. Traceback path

Figure 3: Google Stitch mockups for GeneSync matrix execution and traceback playback



A. Alignment results **B. Hotspots report**

Figure 4: Google Stitch mockups for GeneSync alignment results and hotspot reporting

C. Algorithm Development and Application

The alignment process was developed using dynamic programming because the global alignment problem can be divided into smaller, overlapping alignment problems. Instead of generating and evaluating every possible alignment, the system stores the best score for each pair of sequence prefixes in the scoring matrix. These stored values are then reused when computing larger subproblems.

Let $F(i, j)$ represent the best alignment score for the first i symbols of the first sequence and the first j symbols of the second sequence. The matrix is initialized using

$$F(0,0) = 0, \qquad F(i,0) = i\,g, \qquad F(0,j) = j\,g,$$

where g is the gap penalty. The remaining cells are computed using the Needleman–Wunsch recurrence relation:

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j), \\ F(i-1, j) + g, \\ F(i, j-1) + g, \end{cases}$$

where $s(x_i, y_j)$ gives the selected match or mismatch score for the two symbols. The recurrence ensures that each cell contains the best score among aligning the current symbols, inserting a gap in the second sequence, or inserting a gap in the first sequence.

The Needleman–Wunsch algorithm is used specifically in the matrix initialization, matrix-filling, and traceback processes. These stages solve the main problem of finding the optimal end-to-end alignment between the two input sequences. After the algorithm reconstructs the alignment, the analysis component uses its output to compute percentage identity and gap distribution. In this way, the algorithm provides both the optimal alignment and the structured data needed for the system's statistical and mutational-hotspot reports.

The mathematical output of the Needleman–Wunsch core serves as a structured substrate for downstream diagnostic processing. Let the optimal traceback path be defined as a directed sequence of coordinate pairs. To map structural shifts without human intervention, a **Sliding Window Algorithm** is introduced during the traceback step. Given a fixed window length w and a mutation threshold t, the system shifts a window step by step across the alignment strings and calculates:

$$\text{Density}_w = \frac{\sum(\text{Gaps} + \text{Mismatches})}{w}$$

If $\text{Density}_w \geq t$, the system flags the window bounds as a mutational hotspot.

Furthermore, to evaluate whether an alignment score $F(m, n)$ indicates true biological homology, a **Monte Carlo Sequence Shuffling** model is integrated. The system keeps Sequence 1 static, shuffles the characters of Sequence 2 a fixed number of times, such as $N = 100$, and recalculates the global alignment score for each variation. This generates an empirical random distribution curve, allowing the system to verify that the original alignment score sits significantly outside the boundaries of random chance.

D. Methodology

The project will be developed and evaluated through the following steps:

1. **Parameter and Input Space Definition:** Define the accepted sequence character sets for DNA, RNA, and proteins, and configure the string cleaning parameters, including regex patterns and scoring profiles.
2. **Core Matrix Engineering:** Implement the $O(mn)$ space matrix initialization and the recursive scoring loops driving the Needleman–Wunsch kernel.
3. **Pathfinding and Reconstructive Logic:** Program the single-pointer traceback traversal routine to build the optimal aligned strings from the state matrix.
4. **Analytical Module Integration:** Embed secondary computational frameworks, including the **Sequence Identity Summer**, the **Sequence Shuffling Simulator**, and the **Sliding Window Hotspot Mapper**.
5. **UI Rendering Layout Design:** Construct a dynamic visualization display capable of rendering the dynamic programming matrix cell by cell with a playback path overlay.
6. **Verification and Boundary Testing:** Execute initial validation passes using well-known, brief test strings, such as matching GATTACA vs GTCGACG, to manually audit matrix values against expected outputs. Progress to validating longer target genes downloaded from reference genome databases, such as NCBI or Ensembl GRCh38 gene isolates, up to the system's operational memory boundary cap of 5,000 characters.
7. **Correctness Benchmarking:** Audit the generated scores, execution times, and alignment outputs against established bioinformatics baselines to verify absolute technical accuracy at every pipeline handoff.

For sequences of lengths m and n, the algorithm requires $O(mn)$ time because every matrix cell is computed once. It also requires $O(mn)$ space in this implementation because the complete matrix is retained for traceback and visualization.

VI. References

Choudhuri, S. (2014). *Bioinformatics for beginners: Genes, genomes, molecular evolution, databases and analytical tools*. Elsevier Science.

Fernald, G. H., Capriotti, E., Daneshjou, R., Karczewski, K. J., & Altman, R. B. (2011). Bioinformatics challenges for personalized medicine. *Bioinformatics*, *27*(13), 1741–1748. https://doi.org/10.1093/bioinformatics/btr295

Mount, David W. (2001). *Bioinformatics: sequence and genome analysis*. Internet Archive. https://archive.org/details/bioinformaticsse0000moun_p8z2/page/566/mode/2up

Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, *48*(3), 443–453. https://www.semanticscholar.org/paper/A-general-method-applicable-to-the-search-for-in-of-Needleman-Wunsch/fa24af669bc11711b18ca4f45e9abb85546ed36e

Pearson, W. R. (2013). An introduction to sequence similarity ("Homology") searching. *Current Protocols in Bioinformatics*, *42*(1), 3.1.1–3.1.8. https://doi.org/10.1002/0471250953.bi0301s42